

DevOps

(l'essentiel)

Table des matières

I - DevOps (présentation).....	3
1. DevOps.....	3
1.1. Présentation de devops (origine, concepts, ...)	3
1.2. cycle de vie DevOps.....	6
1.3. Devops , concrètement	9
2. DevSecOps.....	9
II - Essentiel CI/CD.....	10
1. Contexte du DevOps.....	10
1.1. Exemple d'architecture pour environnement de "production".....	10
1.2. Exemple d'architecture pour environnement de "développement" avec intégration progressive.....	13
1.3. Exemple d'infrastructure DevOps.....	14
2. Importance des tests pour DevOps (CI /CD).....	15
2.1. Bonnes idées du cycle méthodologique en V.....	15

2.2. Bonnes idées du cycle "itératif et incrémental"	16
2.3. Principaux types de tests dans les pipelines "CI/CD"	17
2.4. Repères CI/CD	18
3. intégration continue (CI)	19
3.1. Motivation de l'intégration continue	19
3.2. Principe fondamental de l'intégration continue	21
3.3. Rapide historique de l'intégration continue :	22
3.4. Organisation des projets pour l'intégration continue	22
3.5. Build privé/local vs build d'intégration automatisé	24
4. Livraison/déploiement continu	27
4.1. livraison continue (CD)	27
4.2. déploiement continu (CD)	28
III - Annexe – Bibliographie, Liens WEB + TP	30
1. Bibliographie et liens vers sites "internet"	30
2. TP	30

I - DevOps (présentation)

1. DevOps

1.1. Présentation de devops (origine, concepts, ...)

DevOps

Devops est la concaténation des trois premières lettres du mot anglais development (*développement*) et de l'abréviation usuelle ops du mot anglais operations (*exploitation*), deux fonctions de la gestion des systèmes informatiques qui ont souvent des objectifs contradictoires (*ex : nouvelles fonctionnalités apportant de l'instabilité / fiabilité exigée en exploitation*)

Le **devops** est un mouvement en ingénierie informatique et une pratique technique visant à l'**unification** du développement logiciel (*dev*) et de l'administration des infrastructures informatiques (*ops*), notamment l'administration système. (*source wikipedia*)

Origine et principes de DevOps

Apparu autour de 2007 en Belgique avec Patrick Debois, le mouvement Devops se caractérise principalement par la promotion de l'automation et du suivi (monitoring) de toutes les étapes de la création d'un logiciel, depuis le développement, l'intégration, les tests, la livraison jusqu'au déploiement, l'exploitation et la maintenance des infrastructures.

Conférences "DevOpsDays" (la première en 2009 à Gand / Belgique)

Autre terminologie: "**Agile Infrastructure**"

Les principes Devops soutiennent des cycles de développement plus courts, une augmentation de la fréquence des déploiements et des livraisons continues, pour une meilleure atteinte des objectifs économiques de l'entreprise. *(source wikipedia)*

Avant "DevOps"

Univers "Dev" et "Ops" traditionnellement séparés dans les années 1995-2010 pour les raisons suivantes :

- beaucoup de complexités à gérer → besoin de se spécialiser .
- solutions à bases de serveurs (ex : ServApp JEE) à administrer et dans lesquels on déploie des applications (à développer).
- peu d'automatisation , contextes différents (O.S. , ...)

Objectifs "dev"

. apporter les changements nécessaires au moindre coût et le plus vite possible
souvent au détriment de la qualité lorsque des retards viennent mettre les délais du planning en péril

Objectifs "ops"

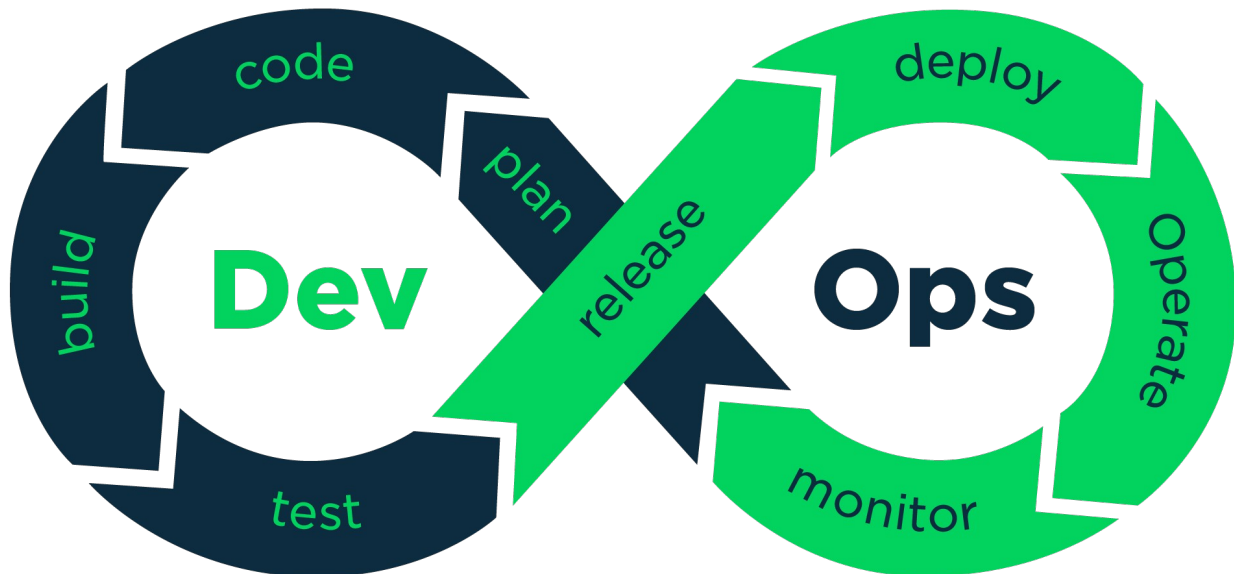
. garantir la stabilité du système.
. se concentrer sur contrainte qualité,
. besoin de temps et de moyens
. contrôler sévèrement les changements apportés au système

Vers "DevOps"

- changement de paradigme ("séparation" → "unification")
- les équipes "dev" et "ops" ne doivent pas se rejeter la faute en cas de problèmes mais doivent collaborer et s'entraider de façon à ne former qu'une seule "méga-équipe" .
- automatisations (intégration continue , déploiements, ...)
- cycles courts (dev, tests, intégration , mise en prod, ...)
- bonnes métriques / supervisions
- applications des méthodes agiles (tests , communications, ...)
- alignement avec les technologies "cloud" et "web"



1.2. cycle de vie DevOps



Phases	Principales activités	Exemples d'outils et technologies
Plan	Planifier, modéliser l'architecture , les fonctionnalités/évolutions , le développement, etc	UML , SCRUM, Trello , Jira, ...
Code	Codage / programmation (avec écriture hyper conseillée de tests unitaires)	IDE (ex : VsCode) + GIT
Build	Construction (éventuelle compilation, éventuel assemblage de librairies , éventuel packaging du code (ex : .jar/.zip , bundle , ...))	maven (java) npm (js/ts) , pip (python)
Test	Tests multiples (unitaire , d'intégration, end-to-end, ...)	JUnit (java), Selenium, Cypress
Release	Si les tests sont ok , générer des versions à mettre en pré-prod	maven+jenkins+nexus ou gitlab-ci , docker
Deploy	Déployer l'application dans un environnement de production	Docker, Kubernetes , Ansible
Operate	Exploitation en production	Ansible, Kubernetes, ...
Monitor	Surveillance du bon fonctionnement (mesures, visualisation, alertes, ...)	Prometheus , Grafana, Nagios

Première phases du cycle "devops"

1. Planification (et modélisation , suivi)



2. Codage/implémentation (avec tests unitaires)



Phases suivantes du cycle "devops"

3. Build (de "dev" ou "release/prod") (assemblage , pré-released)

maven

éventuels profils de 'dev' / 'prod'
avec "bases" et ...
... plus ou moins simplifié(e)s



4. Tests d'intégration (intégration continue)

Exécution automatique des tests dans le but d'avoir un feedback concernant la qualité du code produit en cours de déploiement



JUnit



Phases suivantes du cycle "devops"

5. Release (de ou vers "prod")

maven

profile de 'prod'
avec vraies "bases" / "serveurs" / ...



Environnement au sein duquel l'identification
des anomalies doit idéalement pouvoir se faire aisément.

6. Deploy (packaging & déploiement)



Dernieres phases du cycle "devops"

7. Operate (fonctionnement en "production")



fonctionnement en cluster ,
...

8. Monitoring (surveillance & métriques)

surveillance/exploitation ,
remontées/interprétations des mesures , gestion des logs,
Redimensionnement éventuel (élasticité cloud, ...)

1.3. Devops , concrètement ...

Concrètement "DevOps" c'est avant tout :

- De l'agilité au niveau de l'infrastructure (et pas que sur le papier en théorie)
- Ça se traduit par des automatisations efficaces (intégration continue et déploiement de conteneurs "docker") de façon à ce que les développeurs puissent assez facilement construire des "briques" prêtes à être déployées sans trop de d'ajustements/paramétrages .
- Et inversement , certains informaticiens (très techniques) peuvent packager des environnements sophistiqués (avec sécurité , ...) correspondant aux contraintes réelles de production sous forme d'images "docker" de façon à ce que le développeur puisse facilement y intégrer et tester de nouvelles fonctionnalités.

2. DevSecOps

Le sigle "DevSecOps" est une variante de "DevOps" au sein de laquelle la **sécurité** doit être une **priorité** intégrée tout au long du cycle de vie du développement logiciel.

En version "DevSecOps" on considère que chaque membre de l'équipe est en partie responsable de la sécurité (avec une bonne gestion des "secrets" et où l'on évite évidemment les "mots de passe" en clair) .

II - Essentiel CI/CD

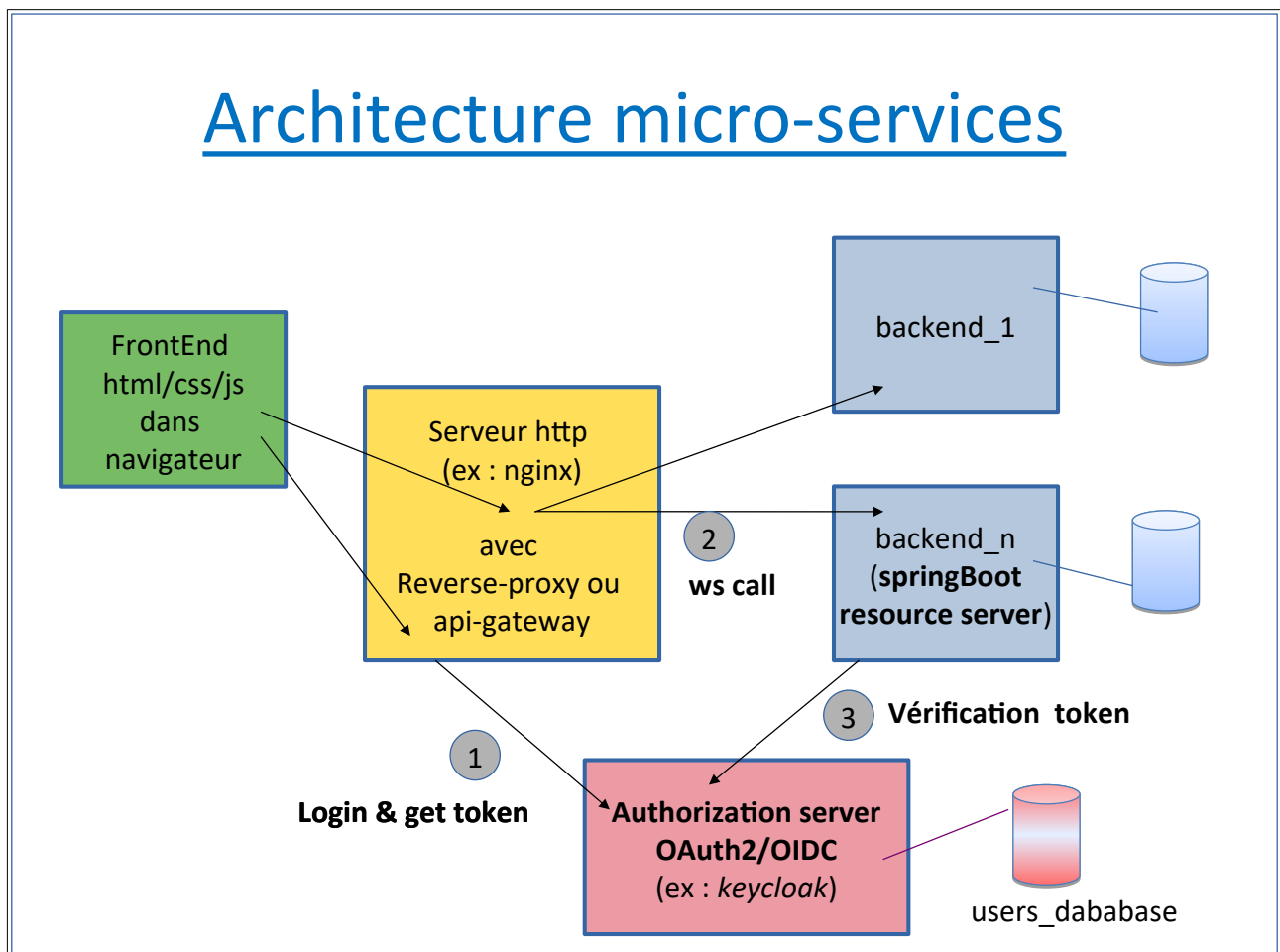
1. Contexte du DevOps

DevOps n'est en soit qu'un ensemble de bonnes pratiques de construction et d'exploitation de logiciels informatiques. Il y a donc **énormément de variantes envisageables dans la mise en oeuvre concrète des pratiques "devops"** .

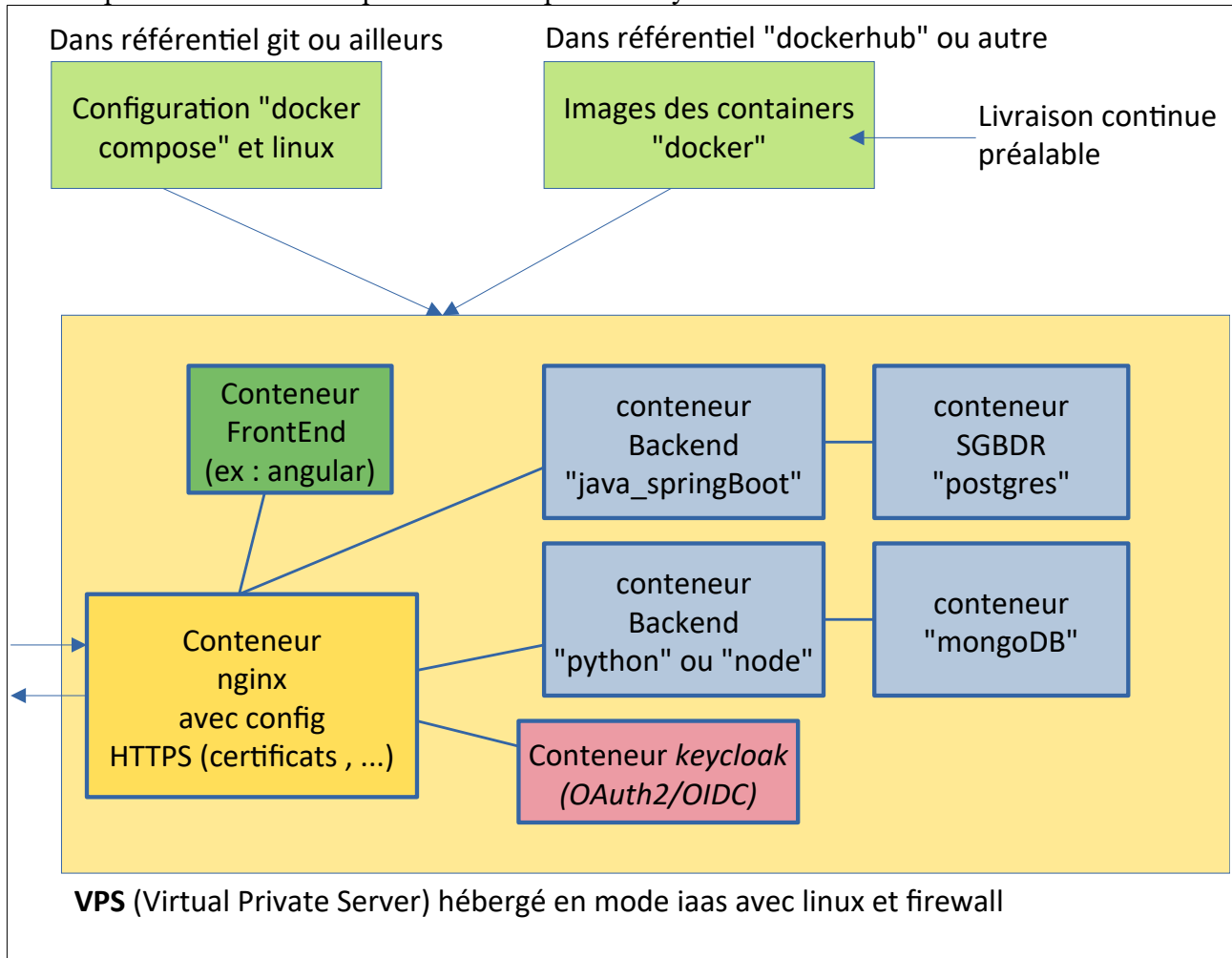
Néanmoins , la plupart des projets informatiques modernes partagent quelques points communs permettant de circonscrire les grandes lignes du cadre "devops" :

- **architectures micro-services** (frontend , backend , database) avec parties souvent développées dans des langages différents (ex : java , python, javascript/typescript, ...)
- code source dans référentiel **git**
- packaging/déploiement du code exécutable via des conteneurs "**docker**"
- cycle de développement itératif et incrémental avec **TDD** (Test Driven Development)
- distinction entre environnements de "dev / mise_au_point" , "production" (et idéalement "pré_prod_integration") + infrastructure "DevOps_CI/CD" .

1.1. Exemple d'architecture pour environnement de "production"



Exemple de mise en production minimaliste en mode IAAS (environ 12euros/mois) sur un seul serveur pour minuscule entreprise avec très peu de moyens :



On peut évidemment facilement mettre en place des variantes d'architectures plus élaborées (par exemple basée sur ansible , docker swarm et plusieurs ordinateur complémentaires) .

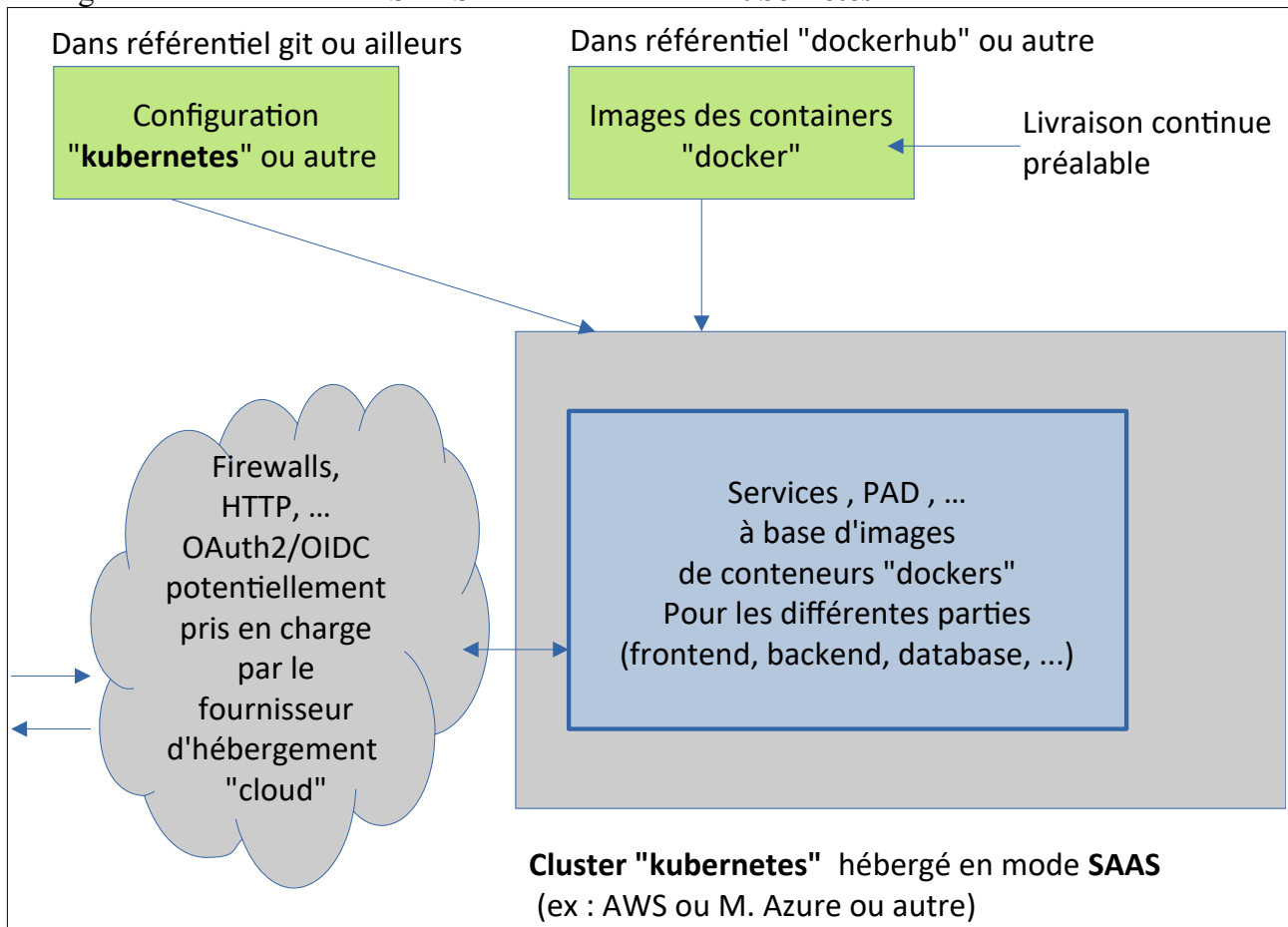
Bien que l'exemple de topologie ci-dessus soit élémentaire, cela montre le minimum à mettre en place pour la mise en production d'une application de gestion un minimum sécurisée (avec firewall , https et authentification outh2/oidc) .

Et pour parvenir à cela , il faut qu'un administrateur récupère deux choses complémentaires :

- des images dockers (normalement préparées par les phases CI/CD)
- une configuration "docker-compose" ou autre

puis qu'il lance quelques scripts "linux ou autres" avec mots de passes à éventuellement saisir (et gardés "secrets").

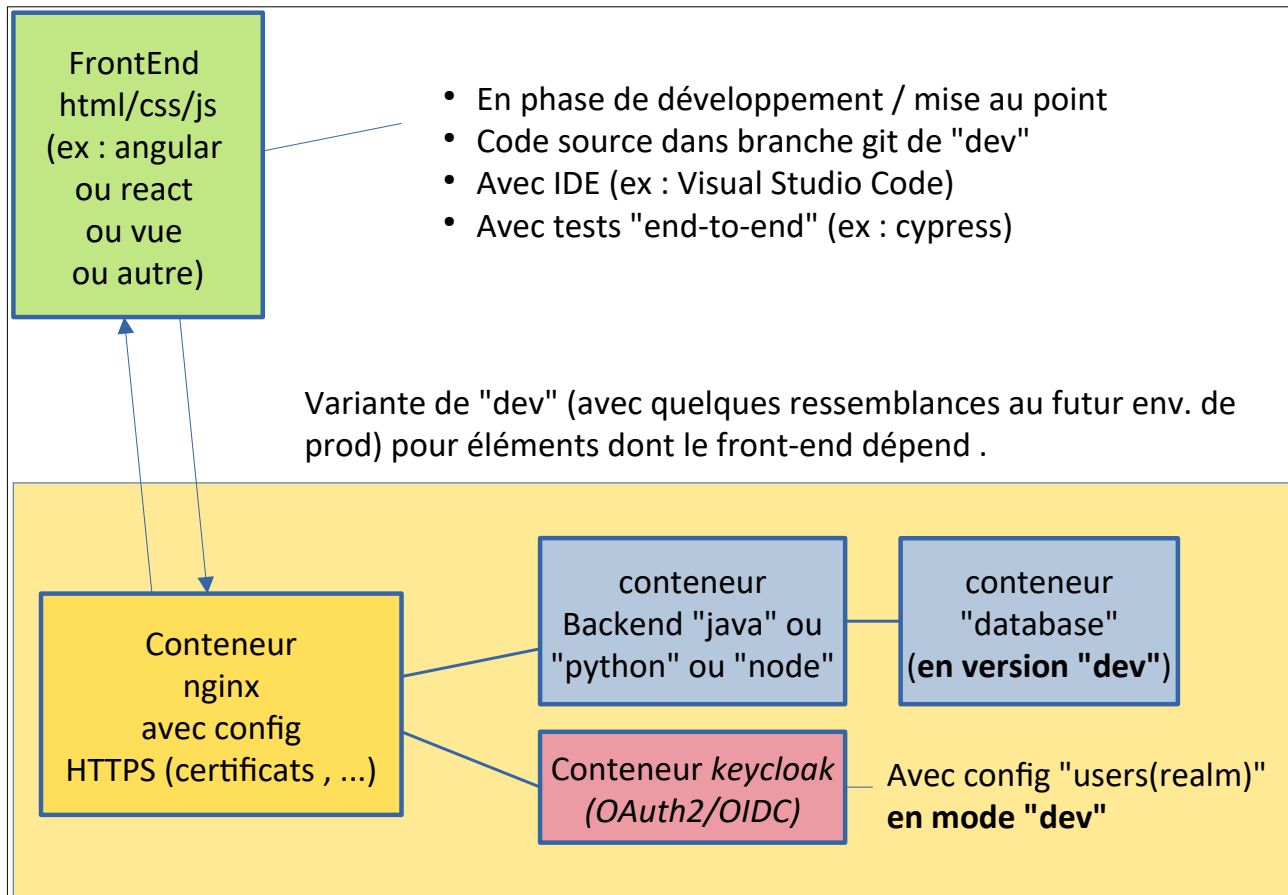
Pour une mise en œuvre robuste de la production on pourra éventuellement s'appuyer sur un hébergement cloud en mode "SAAS" à base de cluster "kubernetes" .



Ordre de grandeur du prix mensuel d'un hébergement cloud/kubernetes auto-géré avec quelques nœuds dans le cluster: quelques centaines d'euros .

En mode "kubernetes" , la configuration change un peu mais les images dockers sont à peu près les mêmes et sont également à préparer/déployer durant les phases CI/CD .

1.2. Exemple d'architecture pour environnement de "développement" avec intégration progressive.



L'exemple ci dessus montre qu'il peut être nécessaire de mettre en place une version de "développement" des éléments en arrière plan de la partie à développer .

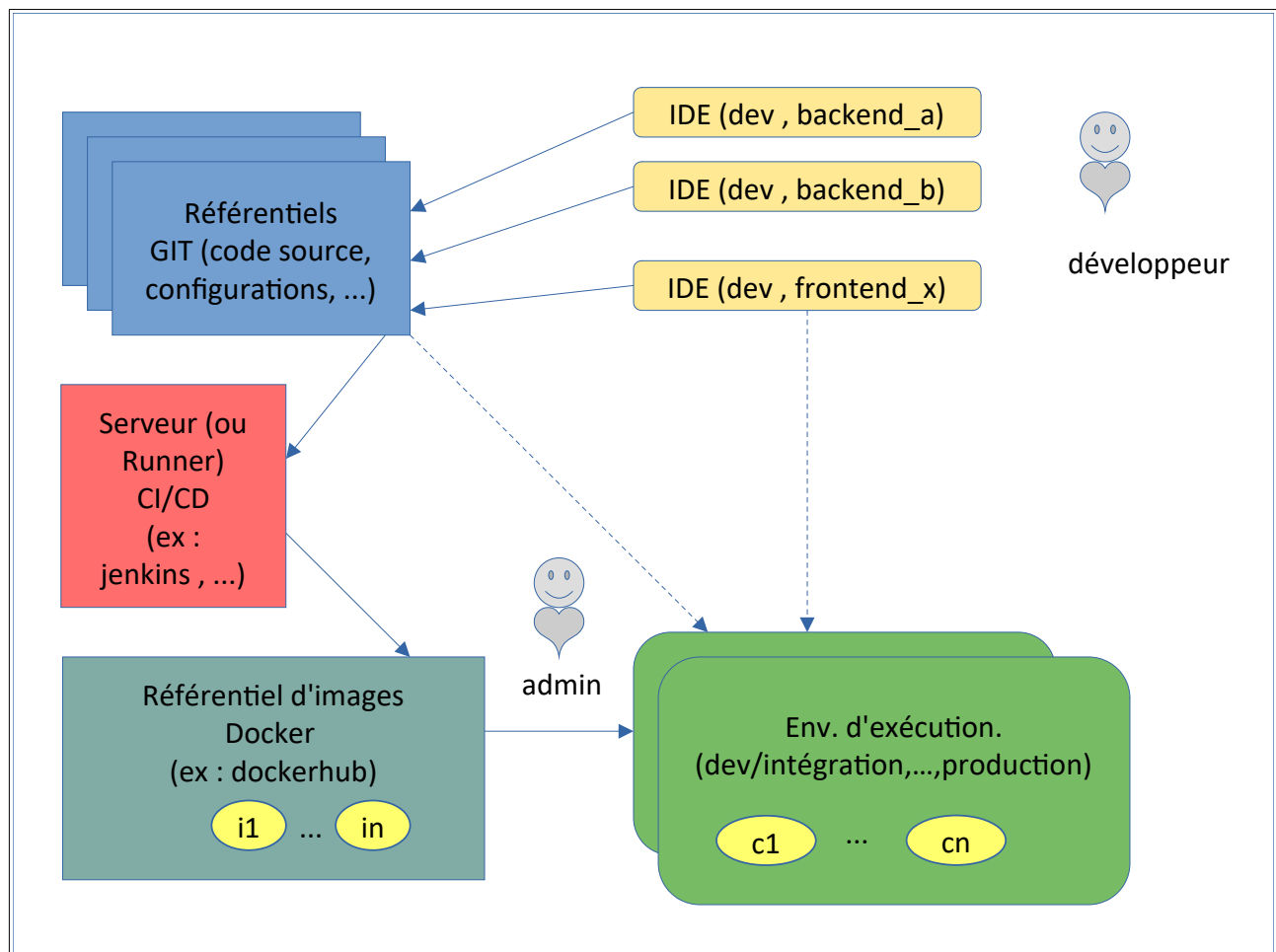
Par exemple , pour tester sérieusement la sécurité d'un frontend ou bien pour effectuer des tests "end-to-end" , on a besoin de pouvoir accéder à "backend + https/oauth2/oidc" ayant un comportement très proche de celui que l'on aura en phase de production (seule les données sont significativement différentes : utilisateurs/passwords , produits , ... en phase de dev) .

Dans le cadre d'un développement classique à plusieurs niveaux de dépendances ($P3 \rightarrow P2 \rightarrow P1$) , on mettra logiquement en œuvre une intégration progressive :

- 1) test P1 en version n1 et intégration de P1 en version n1
- 2) test de P2 en version n2 dépendant de P1 en version n1 puis intégration de P2 en version n2
- 3) test de P3 en version n3 dépendant de P2 en version n2 puis intégration de P3 en version n3
- 4) etc .

1.3. Exemple d'infrastructure DevOps

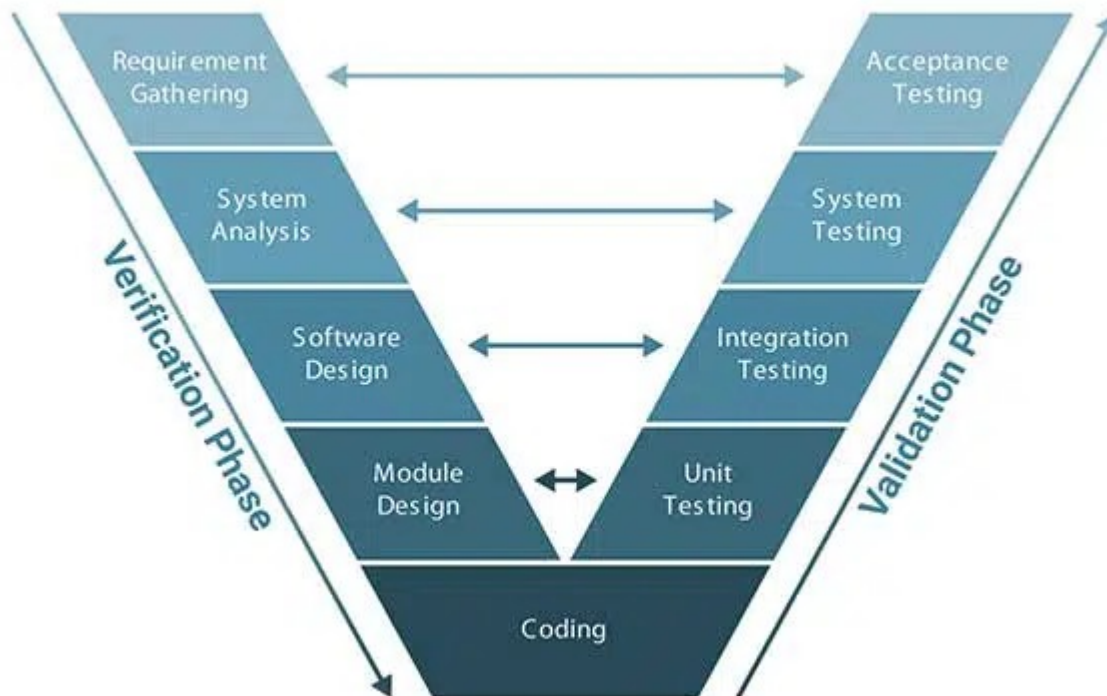
Principaux éléments de l'infrastructure devops (CI/CD) :



<i>Intervenants ou automatismes</i>	<i>Actions</i>
développeur	Écrit composants et tests . Effectue des tests à son niveau (en accédant si besoin à d'autres parties déjà intégrées dans l'environnement d'exécution de dev/intégration) , envoie le code et tests "ok" sur git
Automatismes CI/CD	Récupère du code (avec tests) sur git, lance plein de tests , fabrique et déploie éventuellement des images "docker" si tous les tests sont "ok"
Administrateur	Met au points des configurations (par exemple stockées et récupérées dans certains référentiels git) et supervise le lancement (ou re-lancement) des services applicatifs (conteneurs "docker" ou autres) .

2. Importance des tests pour DevOps (CI /CD)

2.1. Bonnes idées du cycle méthodologique en V

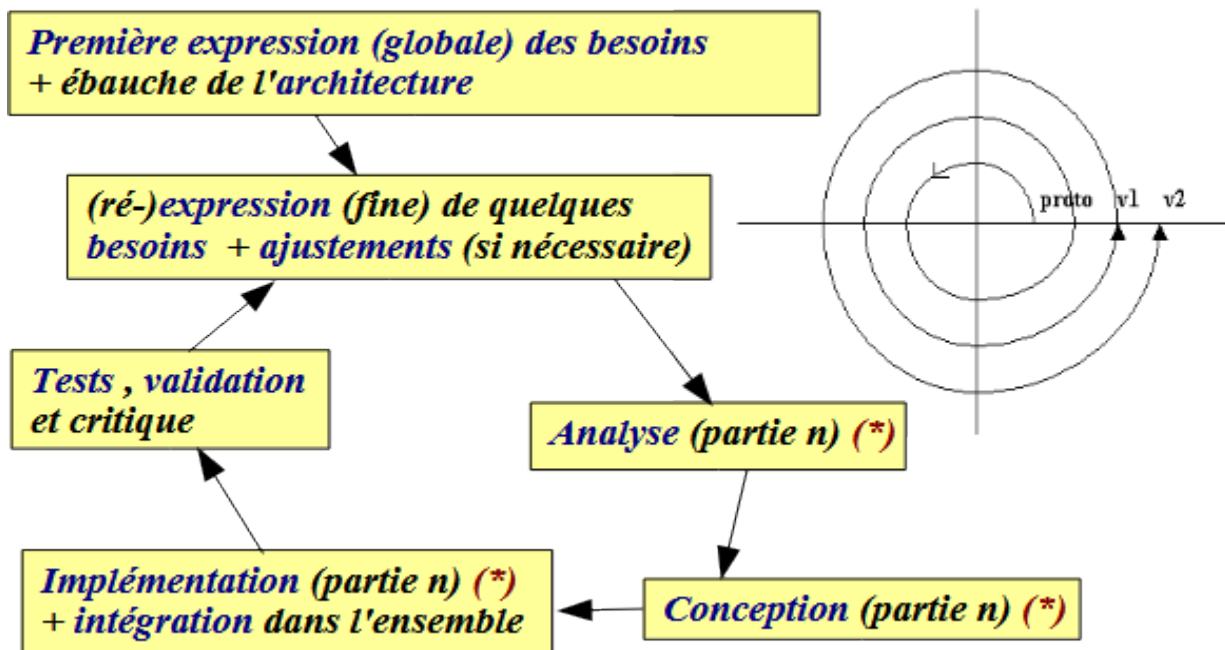


NB :

- "Verification phase" à prendre ici au sens "spécifications + écriture de tests"
- La phase de validation pourra en grande partie être automatisée via une intégration continue en enchaînant les étapes suivantes :
 - compilation et tests unitaires pour valider de fait que chaque partie est "ok"
 - test d'intégration et de niveau système (assemblage des parties + configuration) .
Le tout fonctionne t-il bien sans incohérence (de versions ou autres) dans les parties à assembler ? , "end to end test" ...
 - test d'acceptation fonctionnelle (avec "cucumber" ou autre) : ça respecte bien les règles de gestions?)

2.2. Bonnes idées du cycle "itératif et incrémental"

Itératif et incrémental



() et éventuels correctifs sur parties (1,...,(n-1)) / refactoring*

Le système/logiciel n'est pas conçu/constitué en un seul coup mais en plusieurs itérations.

A la fin de chaque grosse itération :

- nouvelle version à potentiellement mettre en production
- défauts+limitations+fonctionnalités incomplètes qui feront l'objet d'une nouvelle itération.

⇒ dans un cycle devOps, il sera important de versionner :

- le code source placé dans (bibliothèques, backends, frontend, ...)
- les artefacts logiciels générés (bibliothèques compilées, applications, ...)
- les images des conteneurs Docker à livrer et à éventuellement mettre en production

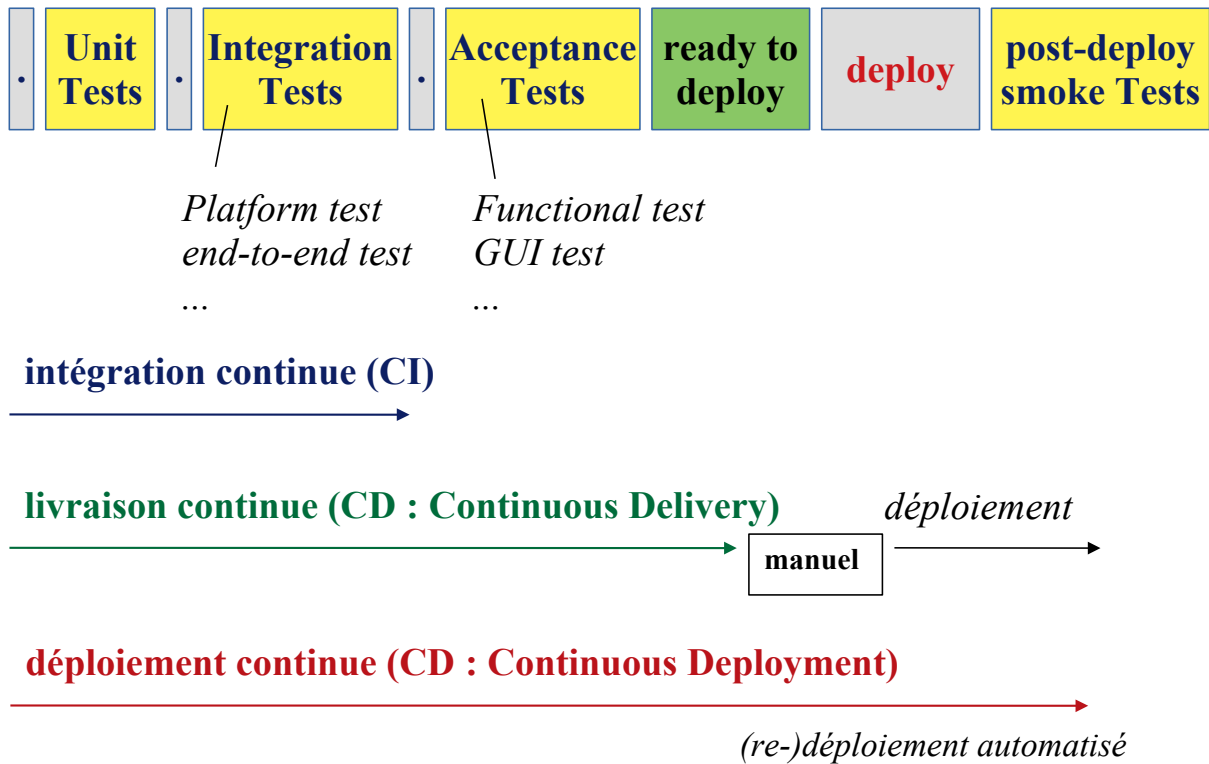
2.3. Principaux types de tests dans les pipelines "CI/CD"

Rappels des différents types de tests:

- Tests unitaires : on teste un seul composant (plus ou moins gros) d'une application en simulant si besoin les autres composants (via des "mocks") .
- Tests de fumée ("smoke tests") : tester si pas de problèmes critiques (ça démarre ? , ça répond ? , ...)
- Tests d'intégration : on teste un assemblage de composants , souvent une application entière .
variante : "end-to-end-test" : frontend+...+backend communiquant bien ?
- Tests fonctionnels : règles métiers bien respectées ?
- Tests d'acceptations : ok pour maîtrise d'ouvrage , ça répond bien aux besoins (perf/sécurité/... , IHM, fonctionnel) ?
- ...

2.4. Repères CI/CD

CI/CD ("Continuous Integration/C. Delivery"):



NB : pour standardiser les phases CI/CD (et éviter trop de différences et cas particuliers en fonction des langages de programmation) , on s'appuiera quasi systématiquement sur les 2 grands standards suivants (qui fonctionnent aussi bien avec java , javascript , php , c/c++ , .net/c# , python, ...) :

- **git** (pour référentiel de code source)
- **docker** (pour packaging des logiciels à mettre en production)

Aux alentours de 2025, les principales technologies CI/CD sont :

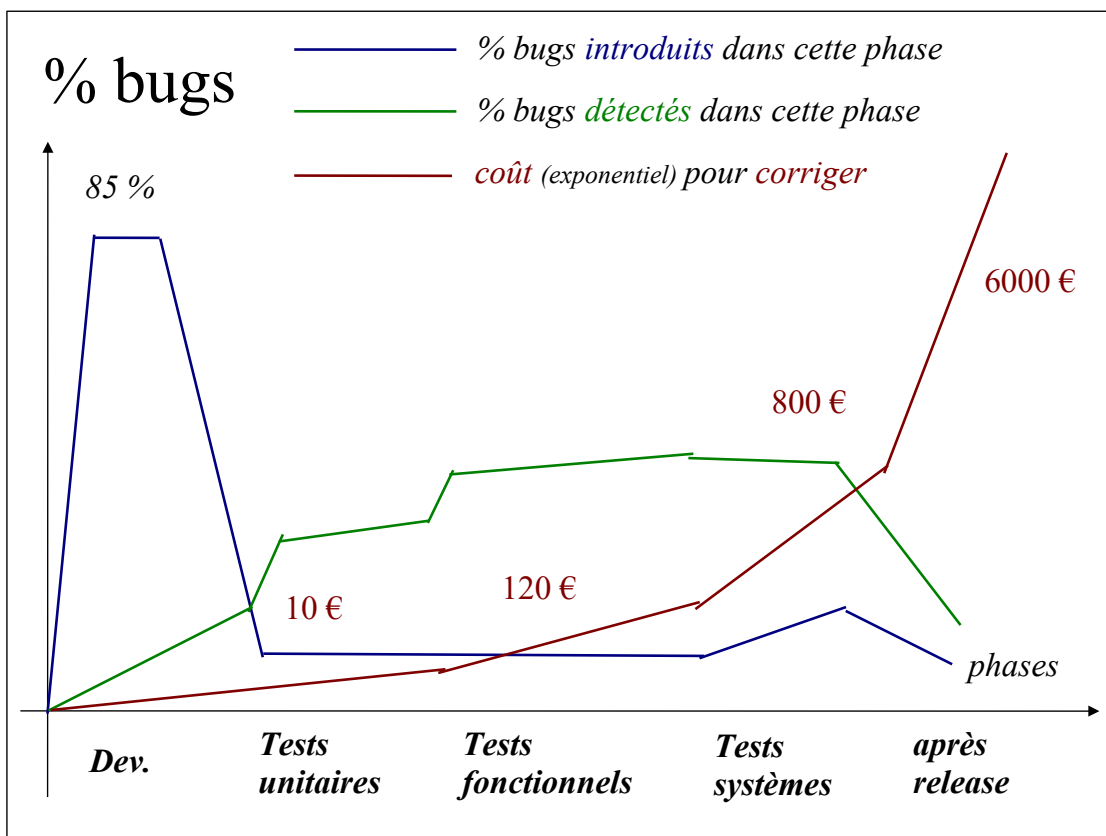
- **jenkins**
- **gitlab-ci**
- **github-actions**

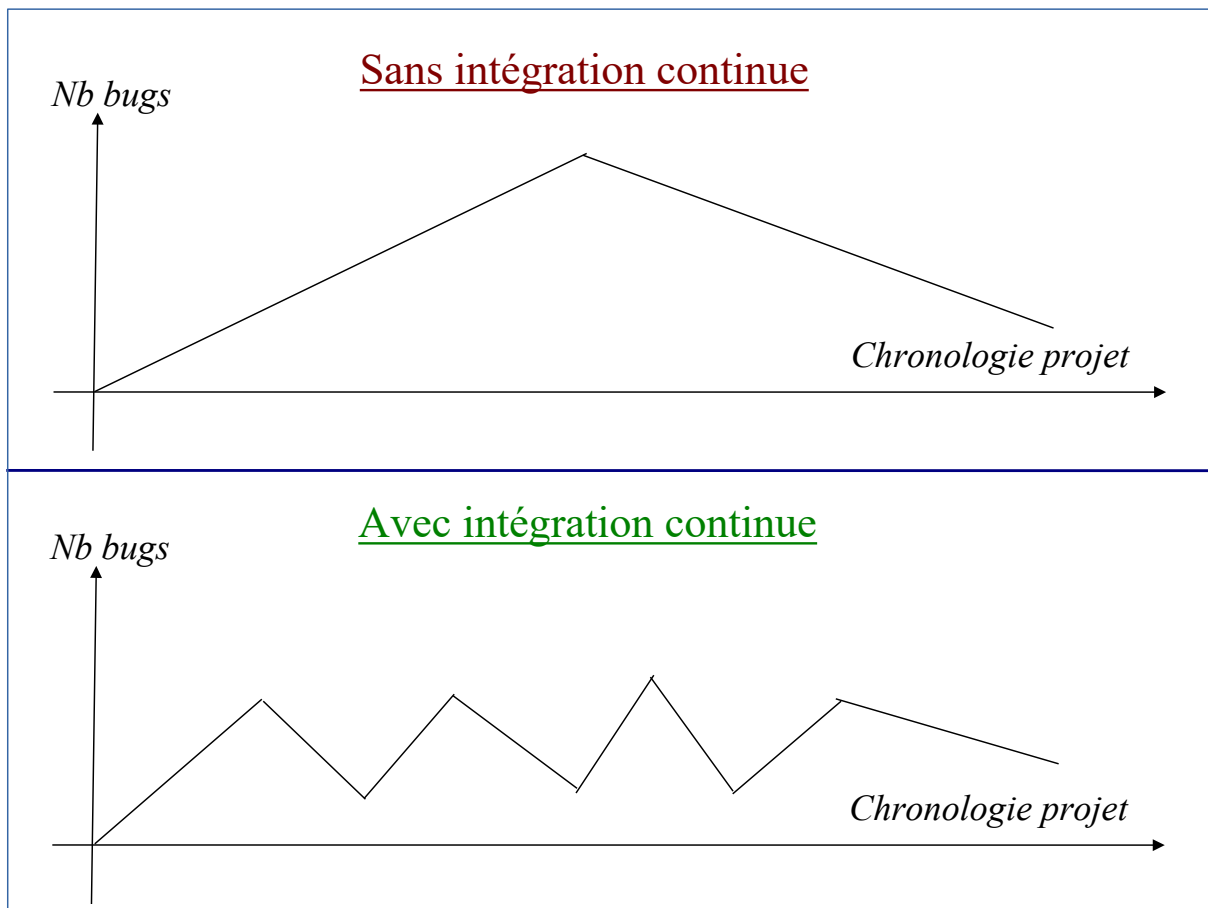
3. intégration continue (CI)

3.1. Motivation de l'intégration continue

Symptômes classiques d'un projet informatique sans intégration continue

Problèmes récurrents	Cause(s) probable(s)	Remède(s) préconisé(s) (I.C.)
Ca marche pourtant sur mon poste	- commits partiels - environnements locaux différents (1 poste avec java 6, 1 poste avec java 7, 1 poste sous windows, 1 autre sous linux). - fichiers de configuration dépendants du poste de travail	- Rebuild systématiquement le tout sur une autre machine en contrôlant bien les paramètres de construction (version jvm,...) - Configuration partagée (avec éventuelles variantes bien gérées via "profils maven")
Sur équipe de 15 personnes, 3 testeurs tout d'abord bloqués (en attente) puis détection d'un bug grave assez tard (après release), construction en urgence d'un patch qui fonctionne bien 3 jours et qui fini par planter lui aussi.	- manque à priori une démarche "pilotee par les tests" - manque d'automatisme et donc : 1) retard en chaîne beaucoup de temps de perdu 2) bugs très coûteux car à résoudre après release (càd avec grande utilisation potentielle)	- Automatiser le plus possible les constructions et les déclenchements des tests pour aller vite - Mettre en œuvre une stratégie pour détecter et corriger les problèmes au plus tôt.





Détecter et corriger les bugs au plus tôt pour minimiser les risques et les coûts

En règle générale, plus une erreur est détectée tard, plus sa correction est coûteuse et d'autre part la plupart des bugs sont introduits durant la phase précoce de développement alors que leurs détections est reportée aux tests.

En automatisant le lancement très régulier et fréquent de toute une batterie de tests, l'intégration continue permet de détecter et corriger au plus tôt un grand nombre de bugs.

L'intégration continue est une des bonnes pratiques préconisées par les **méthodes agiles** dont XP (eXtreme Programming) et SCRUM.

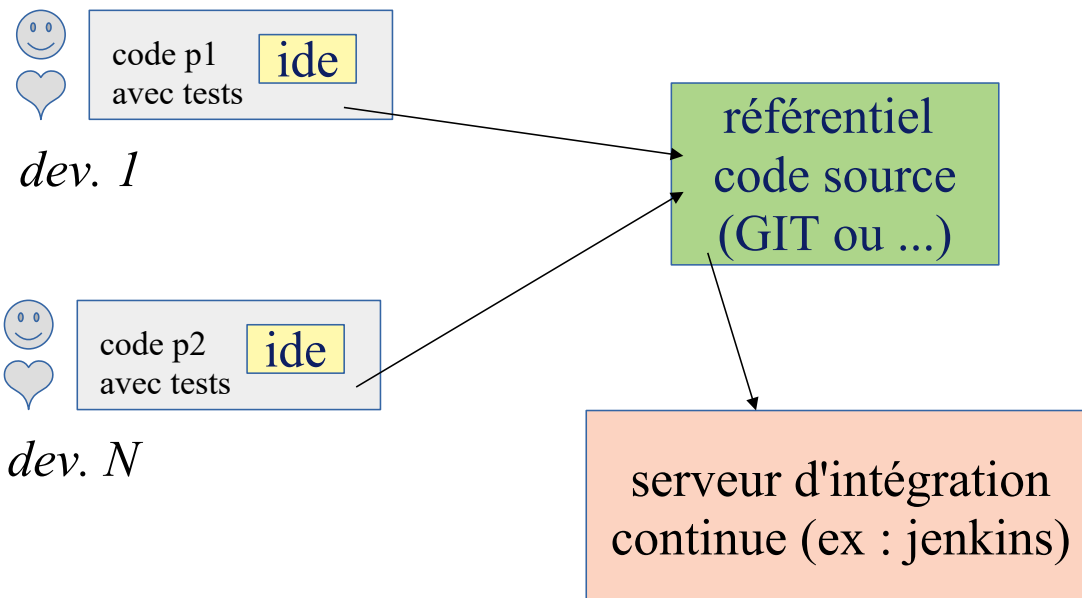
L'**intégration continue** est décrit par son inventeur **Martin Fowler** comme:

* ... UNE **PRATIQUE DE DEVELOPPEMENT LOGICIEL** OU LES MEMBRES D'UNE EQUIPE **INTEGRENT LEUR TRAVAIL FREQUEMMENT**, HABITUELLEMENT CHACUN AU MOINS UNE FOIS PAR JOUR - CE QUI ENTRAINE **PLUSIEURS INTEGRATIONS PAR JOUR**. CHAQUE INTEGRATION EST VALIDEE PAR UN 'BUILD' AUTOMATIQUE (CE QUI INCLUT LES TESTS) POUR **DETECTER LES ERREURS D'INTEGRATION AUSSI VITE QUE POSSIBLE**. CETTE APPROCHE PERMET DE REDUIRE SIGNIFICATIVEMENT LES PROBLEMES D'INTEGRATION ET PERMET A UNE EQUIPE DE DEVELOPPER DES LOGICIELS DE QUALITE PLUS RAPIDEMENT... *

* Extrait (traduit en français) du site <http://www.martinfowler.com/articles/continuousIntegration.html>

3.2. Principe fondamental de l'intégration continue

Intégration continue (cadre typique)



1. Chaque développeur programme sa partie (p1 ou p2 ou ...), la teste de manière unitaire et envoie "code source avec test unitaire réussi" dans un référentiel de code source (ex : git).
2. Le serveur d'intégration continue (ex : Jenkins, GitLab-ci, GitHub-actions, ...) va régulièrement scruter le référentiel de code source et si un changement est détecté, il va automatiquement enchaîner un pipeline d'activités de ce type :
 - récupération éventuellement de la dernière version du code source
 - compilation éventuelle
 - lancement de tous les tests unitaires
 - lancement de certains tests d'intégration ou bien "end-to-end"
3. En cas d'échec lors de la construction/validation de l'assemblage, le serveur d'intégration sera quelquefois capable d'avertir les développeurs de l'équipe (par mail ou par flux rss ou autre).

NB : Une équipe de développement va souvent considérer que le bon assemblage des parties du système est une priorité. Ça ne servirait à rien de développer des parties incohérentes de plus en plus divergentes .

3.3. Rapide historique de l'intégration continue :

Produits des anciennes générations :

- **CruiseControl** – pionnier (dès 2001 , dernière version : 2010)
- **Travis-CI** (à partir de 2011/2012) , n'est plus open source depuis 2020.
- **Continuum** (apache) – dernière version en 2014 .
- **Hudson** renommé **Jenkins** (java+ant , java+maven) vers 2007-2017 .

Produits actuels/modernes :

- **Jenkins** (contrôleur principal + agents , pipeline)
- **Gitlab-ci**
- **Github-actions**

3.4. Organisation des projets pour l'intégration continue

Bien que le client final voit "application globale" , cette application est généralement structurée en plein de parties complémentaires à bien assembler et intégrer .

Pour éviter l'écueil d'un gros projet monolithique mal décomposé , il faudra souvent mettre en place les différents types de projets suivants (chacun dans un référentiel git séparé) :

Type de projets	Caractéristiques et/ou exemples
Librairies réutilisables (facultatif mais pratique)	Fonctions/composants utilitaires , librairies de composants graphiques (pour une technologie frontend de type angular , react ou vue ou autres)
backend	Ex : Appli "SpringBoot" java/maven ou "python" ou "nodeJs" - avec dépendances ou sous configuration (structure base_données ou autres)
frontend	Ex : vanilla_Js ou vue ou react ou angular ou ... - dépendances classiques : plusieurs backends + sécurité + ...
...	
config env. execution	Configuration d'un environnement d'exécution (ex : script ".sh" et/ou "docker compose" ou "kubernetes" ou autre . Ceci en plusieurs variantes (dev/intégration, production, ...)
...	

NB : selon la complexité/taille du logiciel à gérer , on pourra éventuellement mettre en place des projets "d'assemblage/intégration" , etc , ...

D'une manière générale :

- si un bloc de code (ou de configuration) peut être amené à être réutilisé ou partagé → projet à part (avec ses propres versions)
- si une chose n'est pas prévue pour être partagée/réutilisée (ex : structure d'une "base de données" dédiée à un unique "backend" , on peut alors envisager "sous-module" d'un même projet)

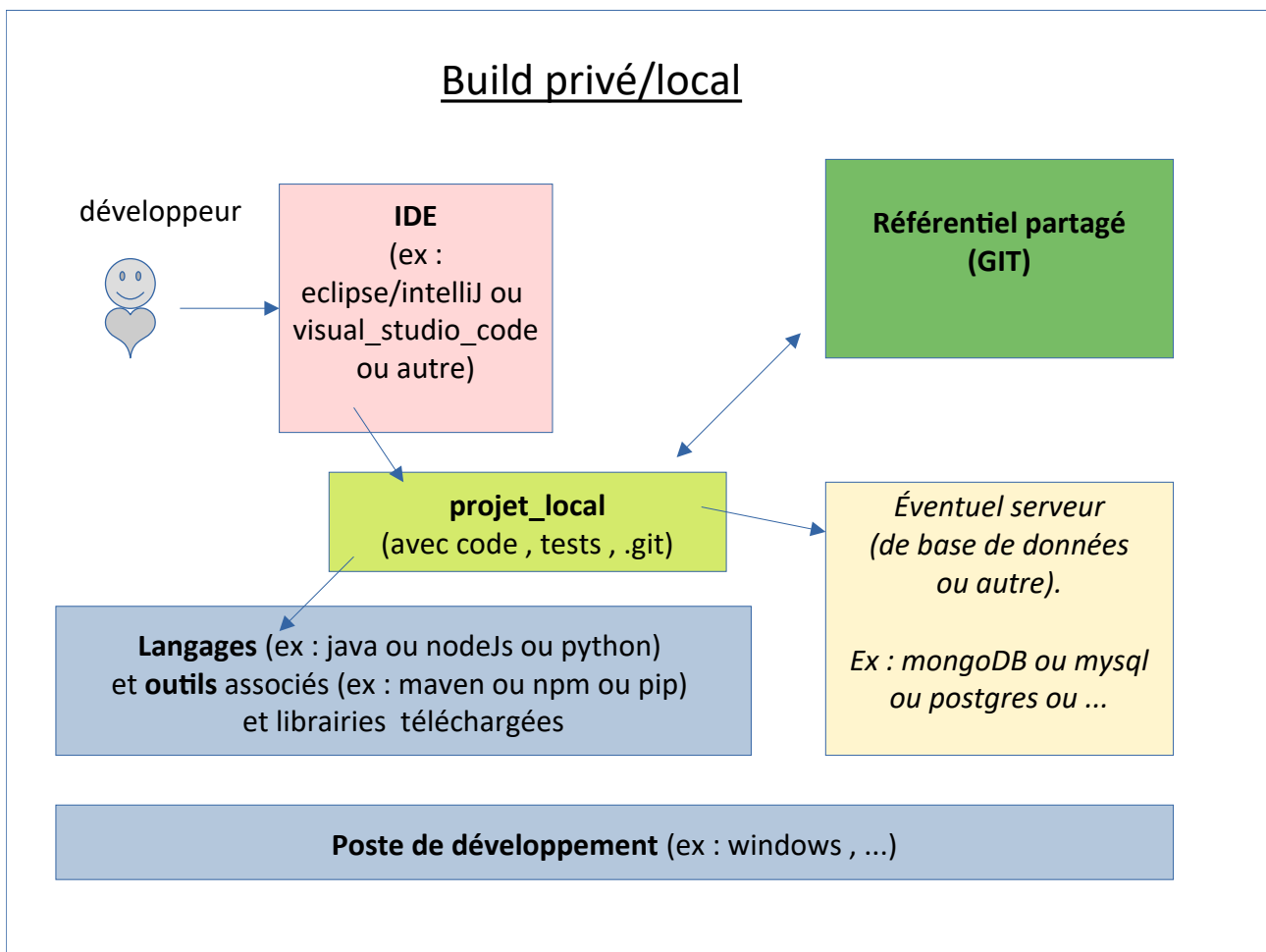
Au sens git , chaque projet pourra comporter plusieurs branches (mais pas trop).

Exemples :

- main ou master (pour la mise en production)
- m-dev (ou autre pour main-dev : branche commune au développeurs d'une équipe)
- f-xyz (branche temporaire de fonctionnalité à mettre au point et intégrer) .

Certains "commit" importants seront tagués (ex : "v0.1" , "v1.2" , ...)

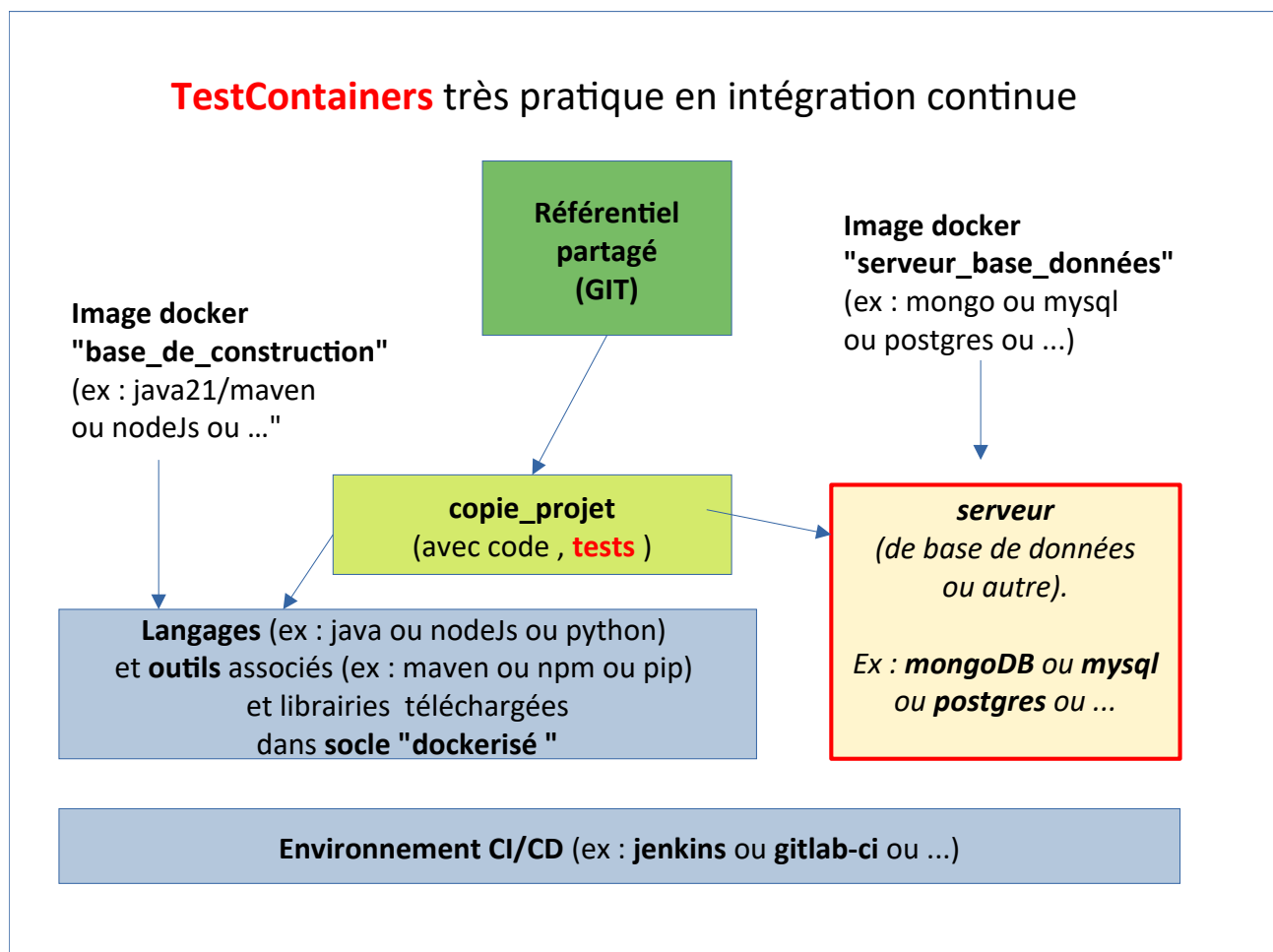
3.5. Build privé/local vs build d'intégration automatisé



Un build privé/local consiste dans la plupart des cas à lancer "éventuelle compilation + packaging + lancement des tests" dans un environnement graphique confortable (ex : IDE sous windows) où beaucoup d'éléments sont installés en local (ex : langage , éventuelle base de données , ...)

Un **build automatisé par une technologie d'intégration continue** pourra éventuellement être déclenché dans un **environnement souple/flexible** (*entièrement en mode texte sans interface graphique*) où la plupart des éléments seront "dockerisés" :

- l'agent jenkins (ou le runner gitlab-ci) sera basé sur une **image ayant le rôle "base_de_construction"** . Cela pourra par exemple être *java21+maven* ou *nodeJs/npm+typescript+cypress* ou *python* , etc .
C'est à partir de cela que sera déclenché le build et les tests (unitaires , intégration) du logiciel à construire (projet courant)
- un **autre conteneur docker** pourra éventuellement être automatiquement lancé en **arrière plan** via la technologie "TestContainers" de façon à **prendre en charge la base de données dont notre logiciel dépend** .



Grâce à "TestContainer" , un code de test d'intégration partielle peut en quelques lignes de code démarrer un serveur de base de données (ex : mongoDB) qui s'exécutera dans un conteneur docker démarré automatiquement.

Autrement dit, pas besoin d'installer préalablement un serveur de base de données au sein de l'agent jenkins ou bien du runner gitlab-ci car tout se fait automatiquement .

NB: la technologie **testContainers** est **disponible** dans **la plupart des langages de programmation** (java/maven , javascript/node/npm , ..)

Exemple d'utilisation de `@testcontainers/mongodb` dans un test javascriptnode/mocha/chai_http :

`xyz.spec.js`

```
import { MongoDBContainer } from '@testcontainers/mongodb'
import { expect } from 'chai'
import mongoose from 'mongoose';

describe("basic mongoContainer test", () => {

  let mongodbContainer=null; //for integration-test (in jenkins or ...)

  before(async () =>{
    if(process.env.TEST_MODE=="IT"){
      try{
        mongodbContainer = await new MongoDBContainer("mongo:8.0.12").start()
        console.log("mdb:"+mongodbContainer.getConnectionString());
        process.env.MONGODB_URL=mongodbContainer.getConnectionString()
      }catch(ex){
        console.log("err start mongodbContainer:"+ex)
      }
    }
  }).timeout(120000);

  after(async ()=>{
    if(process.env.TEST_MODE=="IT"){
      //stop mongodbContainer (in integration test mode):
      await mongodbContainer.stop();
    }
  });

  it("basic mongoose test with @testcontainers/mongodb", async () =>{
    const db = mongoose.createConnection(process.env.MONGODB_URL,
                                          { directConnection: true });

    const obj = { value: 1 };
    const collection = db.collection("test");
    await collection.insertOne(obj);
    const result = await collection.findOne({ value: 1 });
    console.log(JSON.stringify(result));
    expect(result.value).to.equal(obj.value);
    await db.close();
  });
});
```

NB: au sein de l'exemple ci-dessus, `"mongo:8.0.12"` correspond au nom d'une image docker qui sera automatiquement téléchargée/démarrée pour faire fonctionner la base de données dont l'application dépend .

4. Livraison/déploiement continu

4.1. livraison continue (CD)

intégration/livraison continue (cadre et variantes)

référentiel
code source
(GIT ou ...)

serveur d'intégration
continue (ex : jenkins)

référentiel
d'images "docker"
ou autres ...

- tests d'intégration avec ou
sans conteneurs dockers ?

- variantes/profiles de tests
(perf ?, e2e ? , fonctionnel?)

- génération d'artifacts (.jar , ...)
référentiel nexus ou autre ?

- processus d'acceptation ?
(workflow / ...) ?

*éléments livrés (prêts à être
placés en production)*

4.2. déploiement continu (CD)

livraison/déploiement continue (cadre typique)

référentiel
d'images "docker"
ou autres ...

*éléments livrés (prêts à être
placés en production)*

vocabulaire
"provisioning"
= *technologies et
scripts d'installation
& configuration*

- **préparation des machines (vm ou ...)
(installations de logiciels
+ configurations)**
- **déploiement des conteneurs dockers
ou autres (sur cluster "kubernates"
ou autres) , ...**
- **autres tâches d'exploitations
(démarrage , sauvegardes , ...)**

ANNEXES

III - Annexe – Bibliographie, Liens WEB + TP

1. Bibliographie et liens vers sites "internet"

2. TP